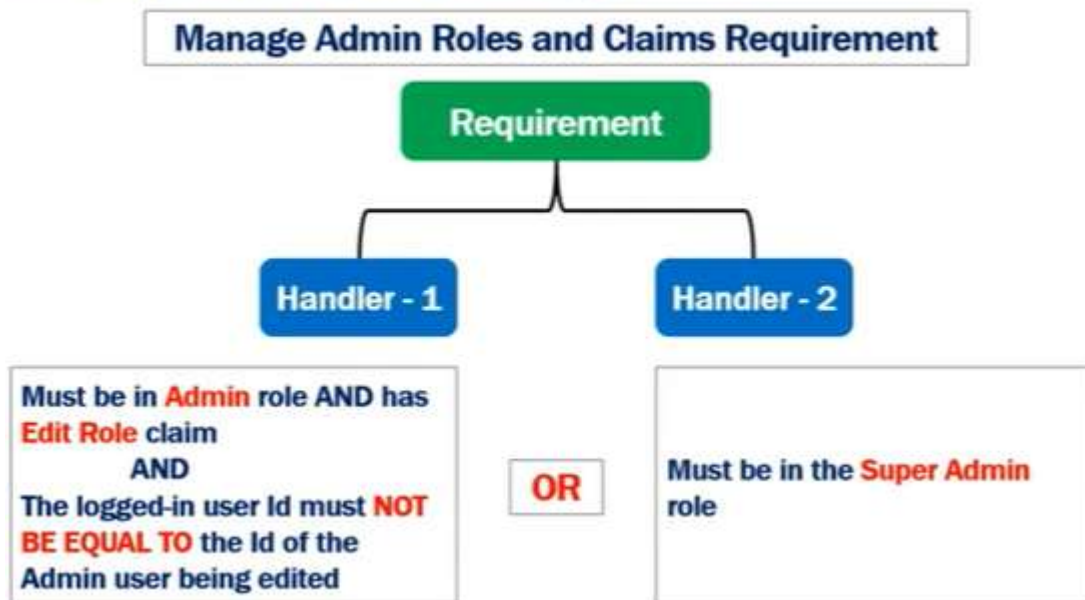


Why Multiple Handlers

We implement **multiple handlers** for a single requirement, when want the requirement evaluation to be on an **OR** basis

Multiple Handlers Example



Към момента имаме само Handler 1, нека направим и 2-я:

Security folder -> new class SuperAdminHandler.cs

Той трябва да унаследява от <T>, където T е типът на изискването по същия начин както в предходния случай:

```
public class CanEditOnlyOtherAdminRolesAndClaimsHandler :  
    AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
```

```

using Microsoft.AspNetCore.Authorization;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    0 references
    public class SuperAdminHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        0 references
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            throw new System.NotImplementedException();
        }
    }
}

```

Всичко, което искаме да изпълни, е само да провери дали потребителят е Super Admin.

```

0 references
protected override Task HandleRequirementAsync(AuthorizationHandlerContext context,
    ManageAdminRolesAndClaimsRequirement requirement)
{
    if(context.User.IsInRole("Super Admin"))
    {
        context.Succeed(requirement);
    }
    else return Task.CompletedTask; Access Denied
}

```

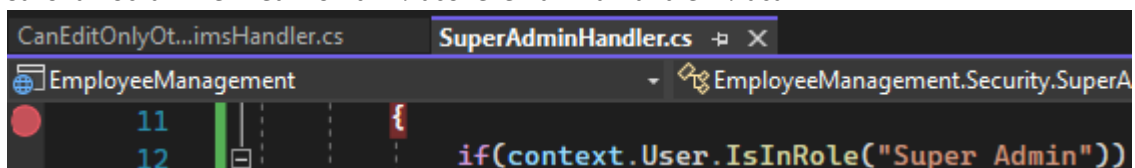
```

using Microsoft.AspNetCore.Authorization;
using System.Threading.Tasks;

namespace EmployeeManagement.Security
{
    public class SuperAdminHandler :
        AuthorizationHandler<ManageAdminRolesAndClaimsRequirement>
    {
        protected override Task
        HandleRequirementAsync(AuthorizationHandlerContext context,
            ManageAdminRolesAndClaimsRequirement requirement)
        {
            if(context.User.IsInRole("Super Admin"))
            {
                context.Succeed(requirement);
            }
            return Task.CompletedTask;
        }
    }
}

```

Когато тестваме в Debug mode обаче, Super Admin не получава достъп до ManageUserRoles, затова поставяме BreakPoint в класовете на 2-та Handler класа:



The screenshot shows the Visual Studio IDE with the file Explorer on the left displaying 'EmployeeManagement' and 'EmployeeManagement.Security.SuperAdminHandler.cs'. The code editor shows the following code with a red breakpoint icon on line 12:

```

11 {
12     if(context.User.IsInRole("Super Admin"))

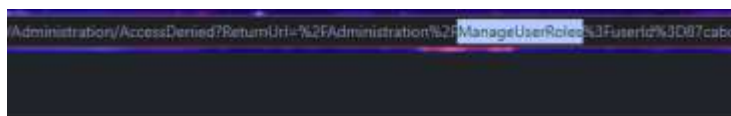
```

```
CanEditOnlyOt...imsHandler.cs  X SuperAdminHandler.cs
EmployeeManagement  EmployeeManagement.Security.CanEditOnlyOtherAdm  HandleRequirementAsync
14 {
15     var authFilterContext = context.Resource as AuthorizationFilterContext;
```

При това положение, при повторно натискане на Manage Roles бутона, BreakPoint of the first Handler is hit.

```
CanEditOnlyOt...imsHandler.cs  X SuperAdminHandler.cs
EmployeeManagement  EmployeeManagement.Security.CanEditOnlyOtherAdm  HandleRequirementAsync
14 {
15     var authFilterContext = context.Resource as AuthorizationFilterContext;
```

We are logged in as Super Admin, so Handler-1 is not valid evaluated, because the logged in user is not Admin + Edit Role claim. Continue -> the second BreakPoint is hit:



But the second BreakPoint is not hit at all. Why? Because we have forgot one of the very important steps: registering the Handler. Stop Debugging -> Startup

```
services.AddAuthorization(options =>
{
    options.AddPolicy("DeleteRolePolicy",
        policy => policy.RequireClaim("Delete Role"));

    options.AddPolicy("EditRolePolicy",
        policy => policy.AddRequirements(new ManageAdminRolesAndClaimsRequirement()));

    options.AddPolicy("AdminRolePolicy",
        policy => policy.RequireRole("Admin"));
});

services.AddScoped<IRepository, SQLRepository>();
services.AddSingleton<IAuthorizationHandler,
    CanEditOnlyOtherAdminRolesAndClaimsHandler>();
services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();
```

```
services.AddSingleton<IAuthorizationHandler,
    SuperAdminHandler>();
```

Даваме отново Debug -> Manage Roles -> BreakPoint Handler-1, BreakpointHandler-2 -> Access.

Manage User Roles

- ☒ User
- ☒ Admin
- ☐ Student
- ☐ Super Admin

При проверка потвърждаваме, че Admin + Edit получава достъп, Super Admin също, но останалите потребители не.